

Universidad Rafael Landívar.
Facultad de Ingeniería.
Licenciatura en Ingeniería en Informática y Sistemas.
Manejo e Implementación de Archivos
Docente: Ing. Manuel Gustavo Rojas Vásquez

INFORME TÉCNICO

“APLICACIÓN DE ESCRITORIO CON GESTIÓN DE CONFIGURACIÓN DE USUARIO”

Estudiante: Velásquez González, Luis Manuel
Carné: 1502325

Quetzaltenango, 15 de septiembre de 2026

1. Introducción y Objetivos del Proyecto

El presente informe detalla el desarrollo y la implementación de una aplicación de escritorio construida en Python, cuyo núcleo funcional es la persistencia robusta de la configuración del usuario. El proyecto resuelve el problema de la pérdida de datos durante procesos de escritura interrumpidos y la falta de resiliencia ante archivos externos manipulados o corruptos.

La solución se fundamenta en la separación de responsabilidades, aislando la lógica de gestión de archivos de la interfaz gráfica. Se implementaron mecanismos de seguridad que garantizan la integridad de la configuración mediante copias de respaldo automáticas y escrituras atómicas, asegurando que la aplicación mantenga un estado consistente incluso ante fallos críticos del sistema operativo o errores de sintaxis en el almacenamiento.

Objetivos Técnicos

- **Persistencia Confiable:** Implementar operaciones de lectura y escritura utilizando el estándar UTF-8 para garantizar la compatibilidad internacional de caracteres.
- **Atomicidad de Escritura:** Desarrollar un flujo de guardado que evite la corrupción del archivo original mediante el uso de archivos temporales y reemplazos a nivel de sistema de archivos.
- **Resiliencia y Recuperación:** Diseñar un sistema de manejo de excepciones capaz de gestionar archivos ausentes, corruptos o sin permisos, permitiendo una degradación segura hacia valores por defecto.
- **Interfaz Reactiva:** Construir una capa de presentación con Tkinter que refleje en tiempo real los cambios en la configuración.

2. Elección y Comparativa Técnica de Formatos de Almacenamiento

Para este proyecto se ha seleccionado el formato **JSON** (JavaScript Object Notation) como el estándar de persistencia. A continuación, se presenta la justificación técnica frente a otros formatos evaluados.

Criterio Técnico	JSON	XML	CSV
<i>Sobrecarga</i>	Baja; utiliza una sintaxis mínima de llaves y corchetes.	Alta; las etiquetas de apertura y cierre duplican el tamaño innecesariamente.	Muy Baja; solo delimitadores, pero carece de metadatos.
<i>Estructura</i>	Jerárquica y anidada; permite objetos complejos.	Jerárquica; muy robusta pero verbosa.	Plana; no soporta relaciones jerárquicas de forma nativa.
<i>Tipado de Datos</i>	Nativo (String, Int, Float, Bool, Null).	Solo texto; requiere conversión manual de tipos de datos.	Solo texto; requiere casting manual y propenso a errores.
<i>Mantenibilidad</i>	Alta; es legible por humanos y fácil de editar manualmente.	Media; la verbosidad dificulta la lectura en archivos grandes.	Baja; difícil de leer si hay muchas columnas o datos extensos.
<i>Soporte en Python</i>	Nativo mediante la librería json. Rápido y eficiente.	Requiere xml.etree o librerías externas más pesadas.	Nativo con csv, pero complejo para datos no tabulares.

Justificación de la elección: JSON ofrece el equilibrio óptimo entre legibilidad humana y eficiencia computacional. Su capacidad para preservar tipos de datos como booleanos para temas oscuros o enteros para tamaños de fuente elimina la capa de validación manual necesaria en formatos como CSV.

3. Arquitectura de Software y Módulos

La aplicación sigue un diseño modular para facilitar el mantenimiento y las pruebas unitarias:

1. **Capa de Persistencia (`configManager.py`):** Encargada exclusivamente del ciclo de vida del archivo. Contiene la lógica de carga, validación, escritura atómica y generación de respaldos. No tiene conocimiento de la interfaz gráfica.
2. **Capa de Presentación (`gui.py / main.py`):** Gestiona la ventana principal de Tkinter, los widgets de configuración y la respuesta visual a las acciones del usuario. Se comunica con el gestor de configuración mediante llamadas a métodos definidos.

Flujo de Datos: Al iniciar, la aplicación desde `main.py` llama al `gui.py` para ejecutar la interfaz visual según los parámetros de configuración que se solicitan al `configManager`. Si el proceso falla, el gestor entrega un diccionario de "Valores por Defecto". Cuando el usuario modifica un parámetro, la interfaz notifica al gestor, quien ejecuta el ciclo de escritura segura en el almacenamiento físico.

4. Implementación de Escritura Segura y Respaldo (.bak)

La escritura directa mediante `open(archivo, 'w')` es considerada una mala práctica para datos críticos, ya que, si el proceso se interrumpe, el archivo queda truncado o vacío. Para mitigar esto, se implementó un flujo de tres pasos:

1. **Copia de Respaldo:** Antes de cualquier modificación, se crea una copia idéntica del archivo actual (`configuracion.bak`) usando `shutil.copyfile`.
2. **Escritura Temporal:** Se genera un nuevo archivo (`configuracion.json.tmp`) con la nueva información. Esto asegura que, si la escritura falla, el archivo original permanece intacto.
3. **Reemplazo Atómico:** Se utiliza `os.replace` para renombrar el temporal al nombre definitivo. Esta operación es atómica en la mayoría de los sistemas de archivos modernos.

Fragmento de Código: Escritura Atómica

```
def escribirConfiguracion(configuracion=configuracionPorDefecto):
    try:
        crearRespaldo()
        with open(TEMP_ARCHIVO, "w", encoding="utf-8") as temp:
            json.dump(configuracion, temp, indent=4, ensure_ascii=False)
        os.replace(TEMP_ARCHIVO, CONFIG_ARCHIVO)
    return True
```

5. Manejo Robusto de Errores y Degradación Segura

La aplicación se diseñó para ser "a prueba de fallos" en los siguientes escenarios:

- **Caso 1: Archivo Ausente (`FileNotFoundError`):** El sistema detecta la ausencia, carga un diccionario predefinido en el código y crea el archivo automáticamente en la primera oportunidad de guardado.
- **Caso 2: Archivo Corrupto (`json.JSONDecodeError`):** Si un usuario edita el JSON y rompe la sintaxis, el bloque `try-except` captura el error, notifica al usuario mediante un mensaje de advertencia y restaura la configuración a valores seguros para evitar el colapso de la GUI.
- **Caso 3: Falta de Permisos (`PermissionError`):** Se maneja mediante capturas a nivel de sistema operativo. Si la aplicación no puede escribir por restricciones del sistema, se muestra un diálogo informativo en lugar de generar un "crash" o cerrar la aplicación.

Saneamiento de Datos

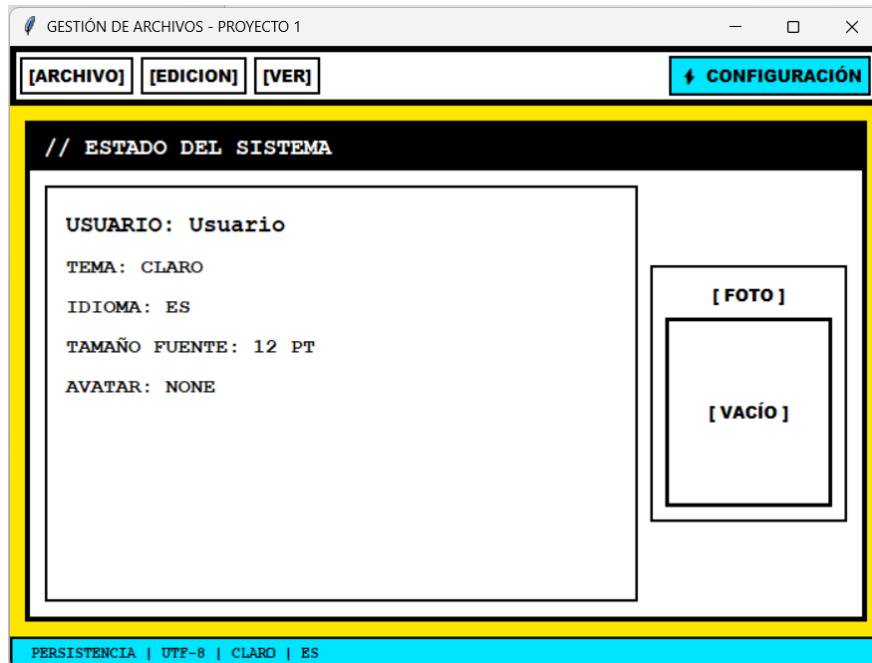
Además de los errores de archivo, se implementó una validación de contenido. Si un valor existe pero es de un tipo incorrecto (ej. un string donde se esperaba un booleano), el `configManager` lo reemplaza por el valor por defecto antes de entregarlo a la interfaz.

6. Codificación de Caracteres (UTF-8)

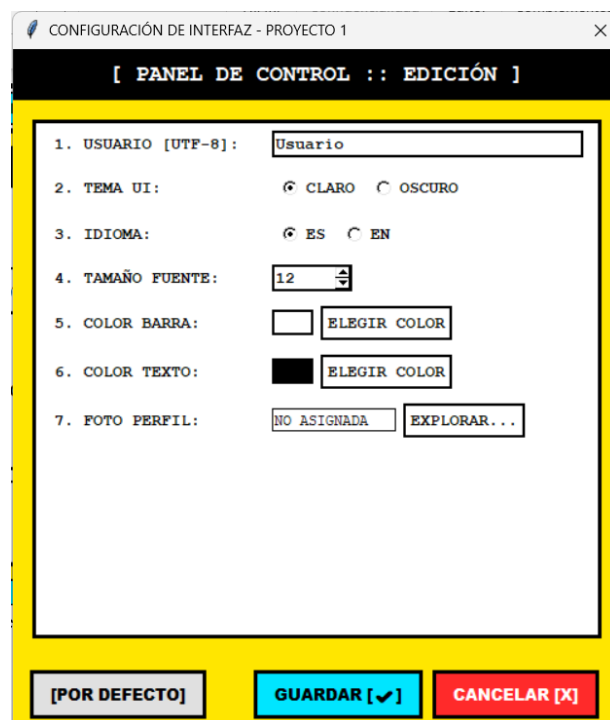
Se ha estandarizado el uso de UTF-8 en todas las operaciones de entrada/salida. Esto es vital para el soporte del español, permitiendo que nombres de usuario o rutas de archivos contengan tildes (á, é, í, ó, ú), eñes (ñ) o símbolos especiales. En Python, esto se logra mediante el parámetro `encoding="utf-8"` en la función `open()` y `ensure_ascii=False` en el serializador de JSON, evitando la conversión de caracteres a secuencias de escape Unicode ilegibles.

7. Evidencias de Funcionamiento

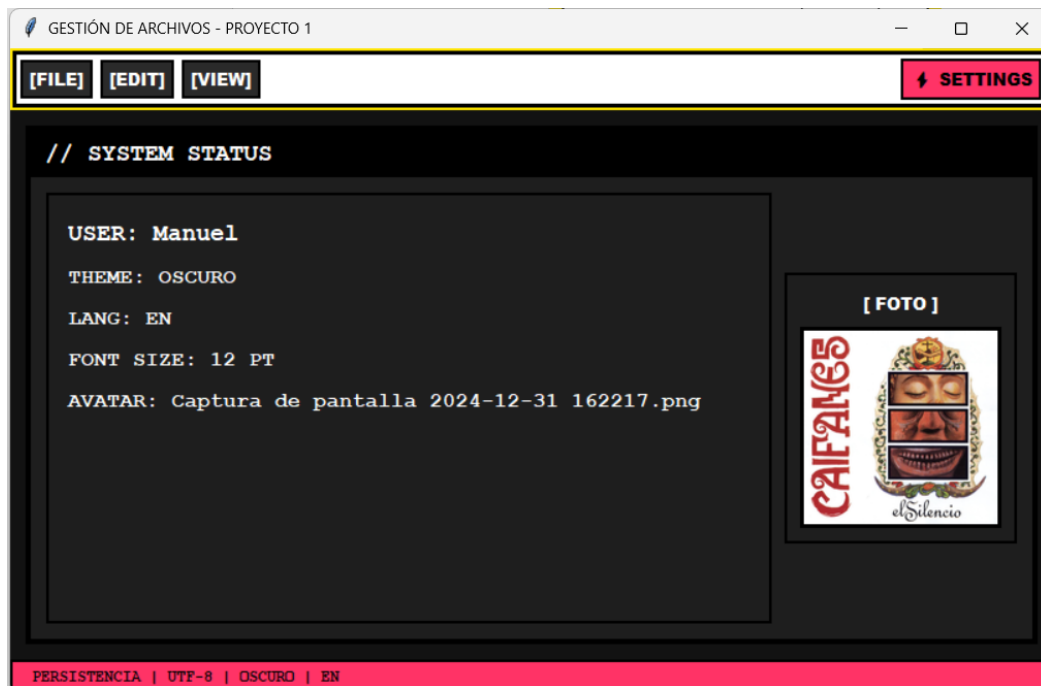
A continuación, se presentan las capturas de pantalla que validan el cumplimiento de los requerimientos técnicos:



Descripción: Muestra la carga inicial exitosa de los parámetros de configuración.



Descripción: Vista de los controles para modificar los datos.



Descripción: Evidencia de la reactividad al cambiar a Tema Oscuro e idioma Inglés.

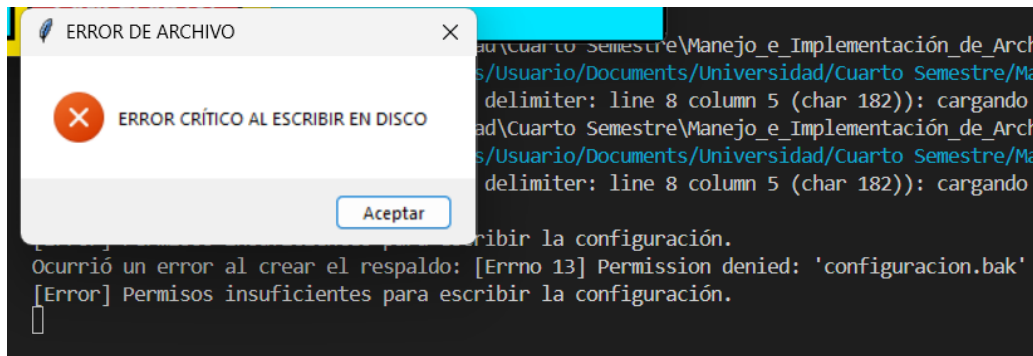
```
except FileNotFoundError:
    print("[Aviso] Archivo ausente: cargando valores por defecto.")
    return configuracionPorDefecto.copy()

ms\Python\Python313\python.exe "c:/Users/Usuario/Documents/Universidad/Cuarto Semestre/Manejo_e_Implementación_de_Archivos/Proyecto 1"
[Aviso] Archivo ausente: cargando valores por defecto.
```

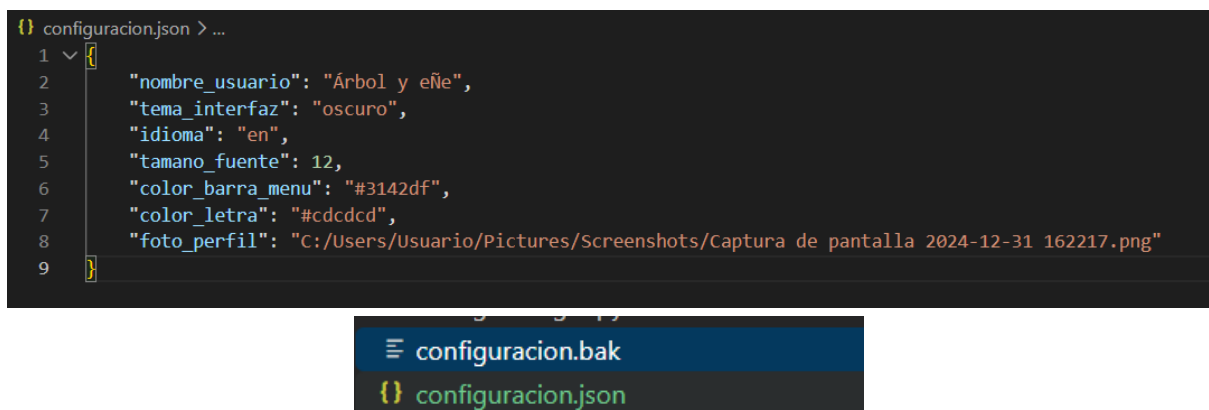
Descripción: Log o mensaje indicando que el archivo no existía y se usaron valores default.

```
PS C:\Users\Usuario\Documents\Universidad\Cuarto Semestre\Manejo_e_Implementación_de_Archivos\Proyecto 1> & C:\ms\Python\Python313\python.exe "c:/Users/Usuario/Documents/Universidad/Cuarto Semestre/Manejo_e_Implementación_de_Archivos/Proyecto 1"
[Error] Archivo corrupto (Expecting ',' delimiter: line 8 column 5 (char 182)): cargando valores por defecto.
```

Descripción: Mensaje de error al detectar un JSON mal formado y posterior auto-reparación.



Descripción: Captura del manejo de excepción ante un archivo con permisos denegados.



Descripción: El archivo configuracion.json abierto mostrando tildes y la existencia del .bak.

8. Ejemplo de Archivo de Configuración Persistido

El siguiente es un ejemplo del estado final del archivo `configuracion.json` generado por la aplicación:

```
{
  "nombre_usuario": "Manuel",
  "tema_interfaz": "oscuro",
  "idioma": "en",
  "tamano_fuente": 12,
  "color_barra_menu": "#FFFFFF",
  "color_letra": "#000000",
  "foto_perfil": "C:/Users/Usuario/Pictures/Screenshots/Captura de pantalla 2024-12-31 162217.png"
}
```

9. Conclusiones y Recomendaciones

1. **Integridad de Datos:** La implementación de la escritura atómica y los archivos temporales reduce prácticamente a cero la posibilidad de archivos corruptos por fallos de hardware o cierres inesperados.
2. **Experiencia de Usuario (UX):** El manejo silencioso de errores (degradación segura) permite que el usuario continúe utilizando la herramienta incluso si los archivos de configuración presentan anomalías, lo cual es superior a mostrar errores técnicos incomprensibles.
3. **Modularidad:** Mantener la lógica de archivos separada de la GUI permitió realizar pruebas de estrés en la persistencia sin necesidad de ejecutar la interfaz gráfica repetidamente.
4. **Recomendación:** Para futuras iteraciones, se recomienda implementar un sistema de suma de comprobación para verificar no solo la sintaxis del JSON, sino que los datos no hayan sido alterados maliciosamente por terceros.